

第九课：多Agent系统理论基础

01

课程主题

项目	内容
课程主题	多 Agent 系统理论基础
课时安排	第5周第1次课（2小时）
讲稿容量	按正常语速约可支撑 180 分钟讲授，可根据授课节奏裁剪为 120 分钟
学员基础	熟悉 Python 编程，理解 Prompt Engineering、Function Calling、ReAct、Agent Skills 与基础工具
教学目标	1. 理解单一 Agent 向多智能体系统演进的工程动因 2. 掌握协作、竞争、协商三类 MAS 核心机制 3. 能分析多Agent系统的典型应用领域与选型边界 4. 能用一个最小案例解释角色拆分、消息流转、状态治理与质量控制

02

环境准备

- Python 3.11+ 环境
- 一个可用的大模型 API Key，支持函数调用或结构化输出更佳
- 可选：AutoGen、CrewAI 或 LangGraph 的基础安装环境，用于下节课衔接演示

03

课程详情

第一部分：课程地图

1.1 开场白

1.1.1 【开场：为什么单个 Agent 迟早会撞墙】

这节课很难，不信我们往下讲。

开头讲这话有点不吉利，但我的目的是希望大家在刚开始听的时候快速交出注意力。

今天这节课偏理论，而且会比较深，比较干。如果大家听不懂，请线下一定多看几遍教案。

到本节课为止，我们讨论的大多数系统，本质上都还能被描述为“一个主 Agent 带若干工具”。这种结构在任务较少、工具数量可控、输出标准单一时非常高效，讲起来和做起来都比较直观。

但一旦任务开始同时要求搜集事实、拆分子任务、并行求解、交叉复核、处理冲突目标时，单 Agent 智能体就会像一个人同时兼任分析师、计划员、执行员、审稿人和调度员。短时间内看起来灵活，长时间运行就会出现上下文拥堵、角色混乱、决策不可追责的问题。此时多 Agent 智能体的出现，解决了单 Agent 在解决复杂任务、多角色的场景所带来的局促感。

此外，还有一个非常贴近工程现场的判断：真正让团队痛苦的，往往不是“这个任务做不出来”，而是“同样的任务今天能做出来，明天换一份材料就开始漂移”。只要系统开始表现出这种不稳定，说明问题已经不只是模型能力不足，而是系统结构对复杂度的承载能力不够了。这个时候 MAS 的现实意义开始凸显。

1.1.2 【虚拟互动：哪些任务已经超出单体 Agent 的舒适区】

我们先看一个非常实用的问题：如果一家公司现在要做一个“研究竞争对手并给出对应策略”的系统，大家会不会只用一个 Agent 从头跑到尾？如果任务再升级为“研究报告 + 数据校验 + 风险审稿 + 行动建议 + 版本复核”，想必大家通常会立刻感觉到，一个 Agent 全包已经不太稳了。

再举两个更具体的例子：

1. 第一个例子是多角色代码协同，一个 Agent 既要理解需求，又要写代码、跑测试、读日志、修缺陷、汇总变更
2. 第二个例子是复杂内容工厂，一个 Agent 既要拆设计需求书，又要出创意方向、调素材、做审核和版本选择。

从以上例子我们隐隐觉得，这些问题并不是模型不能做，而是把这么多不同性质的动作堆到同一个上下文里，系统会越来越像一团没有边界的“聪明黑箱”。

这里真正关键的问题是：在处理这些不同性质的任务里，容易出错的往往不是智能体的产出，而是不知道智能体可能会在哪个环节出问题。这句话非常关键，因为它直接把问题从“模型聪不聪明”转向

“智能体能不能定位责任”。而责任定位，本身就是多 Agent 系统价值的一部分。

1.1.3 【核心洞察：多 Agent 不是“多开几个聊天窗口”】

我们先纠正一个最常见的误解，那就是多智能体 Agent 不是把一个聊天窗口复制成三个，然后让它们彼此说话。那只是形式上出现了多个对话对象，并不代表系统真的有了多主体结构。

真正的多智能体 Agent 至少要回答以下三个问题。

1. 第一，每个角色到底负责什么，边界在哪里。
2. 第二，消息和状态怎么流转，谁能读取，谁能修改，谁负责确认。
3. 第三，拆成多个主体之后，协作带来的收益是否真的大于协调成本。

如果这三个问题答不上来，多开几个窗口只会让系统更慢、更贵、更难调试。

所以今天这节课最重要的认知转变，不是记住几个多智能体名词，而是把多智能体 Agent 看成一种系统分工机制。它和企业组织分工很像：不是人越多越好，而是当目标复杂到一个角色无法稳定完成时，才有必要进行结构化拆分。拆分之后，我们还要管理协调成本、冲突成本和治理成本，否则问题不会消失，只会从“一个人忙不过来”变成“很多人互相打架”。

我们看一个反例。假设一个团队把同一个提示词拆成三个窗口，第一个窗口负责想法，第二个窗口负责补充，第三个窗口负责总结。三个窗口轮流说话，但没有角色边界，没有共享状态，没有版本治理，也没有验收规则。这个系统并没有真正进入多智能体 Agent，它只是生成了一段更长的聊天记录。它看起来有多个主体，实际上没有多主体结构。

落实到真实项目里，判断标准其实也很直接：角色有没有清楚边界，消息有没有固定格式，状态有没有明确归属，出错以后有没有回退路径。四个点都说不清，就不要急着说自己在做多智能体 Agent；只有四个点能说清，系统才有机会变成可设计、可调试、可交付的工程结构。

换成真实项目文档时，这个判断通常会落在三个位置：角色设计说明谁负责，状态设计说明哪些信息共享，验收标准说明怎样才算完成。从这一刻开始，我们讨论的就不是智能体 Agent 数量，而是系统的结构。

1.1.4 【小结：今天这节课要回答的四个问题】

这一小节我们先把整节课的问题框起来。今天我们要回答四个问题。

第一，单一 Agent 的能力边界到底在哪里。也就是说，什么时候一个 Agent 继续往下扛会开始失稳。

第二，为什么系统会从单体执行器演进到 MAS。这里我们不把 MAS 当成一个潮流词，而是把它看成复杂任务倒逼出来的系统结构。

第三，协作、竞争、协商三类机制各自解决什么问题。协作解决分工互补，竞争解决候选方案优选，协商解决约束冲突下的共识形成。

第四，什么样的应用领域适合上多 Agent，什么样的场景最好不要硬拆。这个问题很重要，因为会用 MAS 不等于什么都用 MAS。能判断什么时候不用，才说明你真正理解了它的成本。

这一节最核心的一句话是：多 Agent 不是为了看起来更高级，而是为了在复杂任务中，把能力拆成可分工、可追踪、可评估、可治理的系统结构。

后面每讲一个概念，我们都回到四个词上检查：分工、消息、状态、治理。分工回答“谁负责什么”，消息回答“谁和谁交换什么”，状态回答“系统记住什么”，治理回答“出错后怎么处理”。只要这四个词抓住了，今天的理论就不会变成名词罗列，而会一直和系统设计连在一起。

1.2 课程地图

1.2.1 【过渡：从演进逻辑讲到应用判断】

今天的内容不是先讲框架，再讲概念，而是先讲为什么会出现多 Agent，再讲它靠什么机制成立，最后再回到实际场景判断值不值得用。

这个顺序非常重要，因为如果一上来就讲框架名词，大家容易记住工具，却忘记问题本身。

这里要强调的是：今天不是在学一套“更炫的调用方法”，而是在学一套更适合复杂任务的系统组织逻辑。只要逻辑抓对了，后面的 AutoGen、CrewAI、LangGraph 才会变成工具选择，而不是宗教站队。

这套顺序有一个好处：先把问题讲清楚，再讲机制，最后讲落地。否则一开始就谈框架，很容易把多 Agent 理解成工具配置；先理解问题，再看工具，后面的学习路径才不会跑偏。

下面我们介绍下本课地图。

1.2.2 【第一站：从单一 Agent 到 MAS 的演进】

第一站我们先回答“从单一 Agent 到 MAS，为什么会演进”。我们会看到，MAS 的出现并不是理论爱好者的抽象兴趣，而是复杂任务、长链路执行和多目标约束共同推动的结果。换句话说，MAS 是工程压力逼出来的，而不是概念营销吹出来的。

这一站讲完后，大家应该能建立一个非常朴素但很重要的判断——那就是系统结构的升级，往往是由失败经验倒逼出来的。单体结构在简单问题上并不落后，只有当任务结构开始持续击穿它的稳定边界，分工才会变成必要，而不是可选装饰。

我们如果把系统发展看成一条线，单体 Agent 解决的是“一个主体如何完成任务”，而 MAS 解决的是“多个主体如何共同稳定完成任务”。

我们必须明确，这不是能力大小的差别，而是组织方式的差别。

1.2.3 【第二站：协作、竞争、协商三大机制】

第二站我们进入理论核心。前面我们讲过了，协作解决的是分工互补，竞争解决的是候选方案优选，协商解决的是冲突约束下的共识形成。三者不是互斥关系，而是多 Agent 系统中最常见的三类运行逻辑。

技术路径上我们可以这样理解：如果只会协作，我们能做出流程型系统；如果会用竞争，我们能做出更有比较能力的系统；如果再加上协商，我们能真正触及更加复杂的组织系统。这个层次感一旦建立，大家后面看案例时就会更有抓手。

原理部分我们再看一个反例：如果没有这个机制，系统会如何退化。比如没有共享状态就会彼此失联，没有评分机制竞争就会变成多采样，没有硬约束，协商就会突破底线。这个反例能帮助大家记住概念边界。

1.2.4 【第三站：应用领域与选型边界】

第三站我们不按行业先看，而按任务结构看。研究分析、代码协同、流程调度、资源分配、仿真博弈，这些场景看起来不同，但如果都具有多角色、多目标、多状态的特征，就有可能适合 MAS。反过来，如果任务单步可解、收益链太短，就不该强行上多 Agent。

还要提醒一点：应用分析的目的，不是给 MAS 找尽可能多的落点，而是帮团队学会做减法。越是新技术热的时候，越要能说明哪些需求不值得做，哪些场景不该拆，这种克制能力反而更体现工程成熟度。

讲到风险时，我们要抓住“早期信号”：如果系统开始依赖很长的补丁提示、错误定位越来越慢、每次返工都要重跑整条链，就说明结构风险已经显性化。此时应优先调整边界和状态，而不是继续加描述词。

换成真实项目决策，最重要的不是先问“这个行业能不能用 MAS”，而是先问“这个任务里有没有稳定存在的多个责任主体”。行业只是外壳，任务结构才决定架构。

1.2.5 【终点站：用一个最小案例把理论落到设计】

最后我们会用一个轻量案例，把今天的理论落成一个最小多 Agent 系统，我们会介绍角色怎么拆、状态怎么流、消息怎么传、输出怎么验收、以及为什么它比单体 Agent 更稳。这样后面我们介绍 AutoGen 时，大家就不会把框架当魔法，而是知道它到底在替我们管理什么。

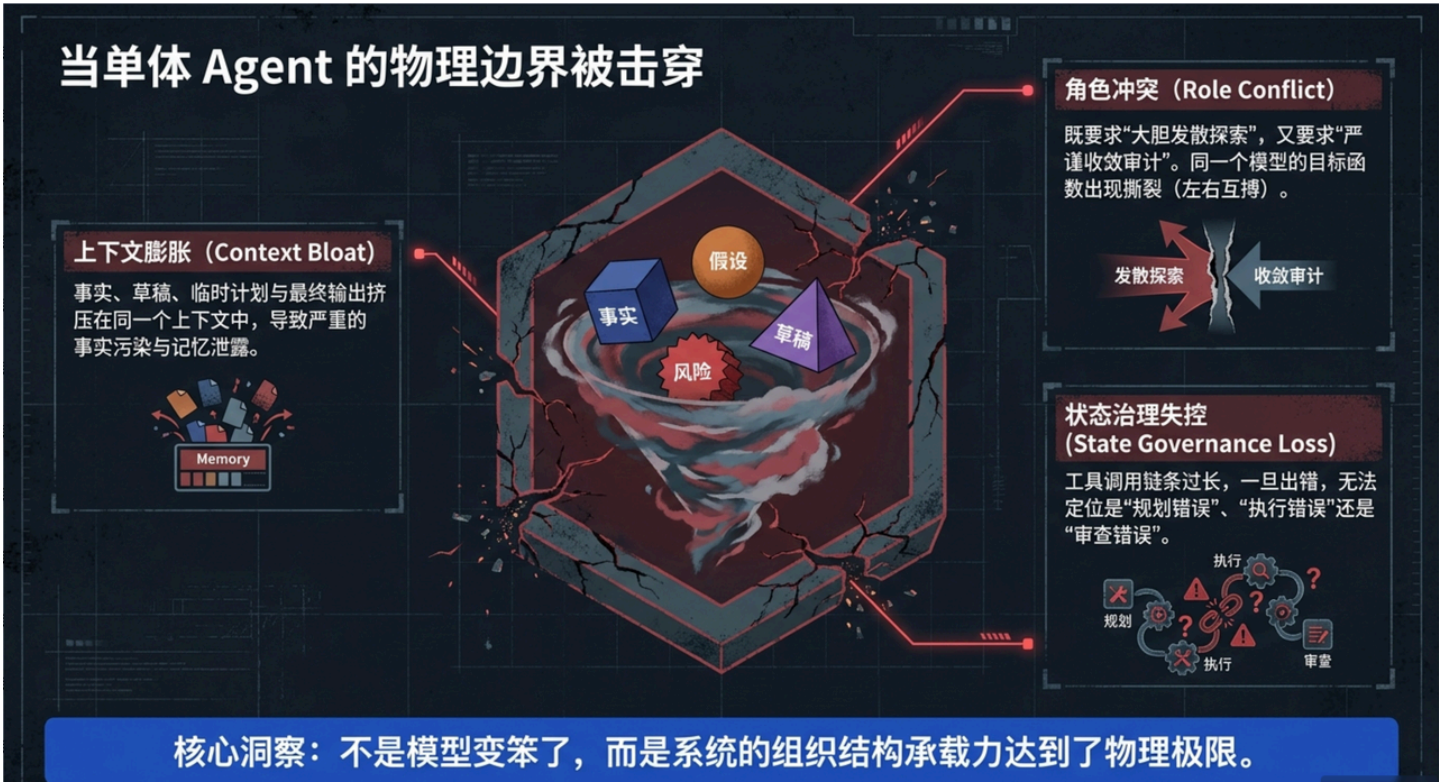
这里先建立一个预期：这个最小案例不是为了展示“花哨的多轮对话”，而是为了让看到看到一个复杂任务被拆成多个责任边界之后，为什么更容易解释、复盘和持续迭代。只要这个目标讲清楚，案例部分就会自动和理论部分扣合起来。

第二部分：从单一 Agent 到 MAS 的演进

2.1 单体 Agent 的能力上限

我们先介绍单体 Agent 能力上限。

这一段我们承接上面的演进逻辑，并把讨论引向后面的机制和案例。



2.1.1 【开场：单体 Agent 在什么条件下已经足够好】

先把边界说清楚，避免大家误以为今天是在否定单体 Agent。事实上，大量真实任务并不需要 MAS。只要任务目标单一、上下文规模有限、工具调用次数不多、失败成本可接受，一个主 Agent 加若干工具的结构往往已经是最优解。它简单、可控、便宜、调试路径短，这是巨大的工程优势。

所以多 Agent 理论的前提不是“单体不好”，而是“单体何时不再划算”。只有当复杂度上升到单体结构开始失稳时，MAS 才具有讨论价值。这个判断非常关键，因为如果没有“单体优先”的意识，团队很容易在还没必要的时候就把系统拆碎，提前背上治理负担。

这里最好再补一个工程启发：成熟团队的默认姿势往往是先把单体 Agent 做到足够稳，再去证明它为什么不够。因为只有先把简单结构做到位，你才知道真正逼迫你升级的瓶颈来自哪里，是上下文、是责任边界、是质量控制，还是资源调度。没有这一步，很多“多Agent需求”其实只是基本功没做好。所以在工程判断上，单体 Agent 并不是落后方案。很多时候，越简单的结构越可靠。只要它能稳定完成任务、能定位错误、能控制成本，就没有必要为了形式上的先进感去拆分。

2.1.2 【短板一：上下文窗口与任务链膨胀】

单体 Agent 的第一个短板，是上下文与任务链同步膨胀。一个任务如果既要读取长文档，又要查外部数据，还要形成中间计划、维护临时状态、记录审查意见，那么上下文里会同时堆积事实、假设、工具结果、待办事项和最终输出草稿。上下文越长，角色越混，模型越容易在重要信息和临时噪声之间反复切换。

这类问题不是简单靠“更大的上下文窗口”就能彻底解决。窗口变大只意味着系统更能装，不意味着系统更会组织。真正的难点在于，不同类型的信息是否被正确分层、不同阶段的任务是否被正确隔离。单体 Agent 一旦既当计划器又当执行器又当审稿人，就容易把多个认知阶段压扁到同一层里，进而让错误沿链路向后传播。

让我们来看一个非常具体的失稳现象：系统前面刚刚根据资料 A 得出假设，后面又在新工具返回的数据里看到与之冲突的事实，但由于没有显式状态分层，模型会把旧假设和新事实混写进同一份草稿，最后生成一个“语气很自信但内部互相打架”的结果。这正是上下文膨胀在工程上最常见的症状。

上下文不是容量问题，而是组织问题。把事实、假设、工具返回、中间计划、最终草稿分开看，让大家看到同一段上下文里混入不同性质信息后，模型会把“已确认事实”和“临时判断”混写在一起。这里可以拆成四层：事实层、计划层、执行层、审查层，四层没有隔离时，错误会沿链路向后扩散。

2.1.3 【短板二：角色冲突与目标混杂】

单 Agent 的第二个短板是角色冲突。比如同一个 Agent 既负责“尽可能多找信息”，又负责“尽快输出结论”，这两个目标天然就可能互相拉扯。再比如同一个 Agent 既要大胆提出方案，又要保守评估风险，也会出现价值尺度不稳定的问题。系统外面看起来只有一个回答，系统里面其实已经混杂了多个不兼容的评价标准。

从组织理论角度看，这相当于把探索、执行和审计三种角色压在一个脑子里。探索要求发散，执行要求收敛，审计要求怀疑。把它们硬塞进一个上下文中，短期看省事，长期看会导致系统既不够发散，也不够严谨，最后只能靠提示词不断补丁化。补丁越多，系统可解释性越差。

再补一个更直白的说法：让同一个 Agent 既“大胆创新”又“严格审计”，就像让同一个人一边做销售承诺、一边做法务复核，理论上不是不能做，实践上却很容易在不同角色逻辑之间摇摆。角色冲突不是语言表面的问题，而是目标函数混杂的问题。这恰恰是 MAS 通过角色隔离最想解决的东西。

这里要把“角色冲突”讲成目标函数冲突，而不是人格设定冲突。同一个 Agent 同时被要求大胆发散、快速收敛、严格审稿时，会在探索、执行、质疑三种认知动作之间摆动。可以用产品方案评审举例：写方案的人天然倾向证明方案可行，审稿人天然要找证据缺口，把两种动作放在同一个上下文里，常常会让系统既不够大胆，也不够严格。

2.1.4 【边界三：工具调用与状态治理成本】

单 Agent 第三个短板是工具和状态治理。单体 Agent 调一个工具、拿一个结果、给一个回答，这很容易理解。但当工具链升级为“搜索、抓取、解析、汇总、打分、复核、写入”这种长链路时，系统就不仅仅是在做推理，而是在管理状态。哪些结果是中间态，哪些是最终态，哪些可以丢弃，哪些必须保留，都需要明确约束。

如果这些约束仍然只放在一个 Agent 的提示词里，系统会开始出现两类问题。第一类问题是状态污染，前一步的噪声会直接影响后一步判断。第二类问题是责任模糊，出错之后很难定位究竟是计划阶段、执行阶段还是审查阶段出了问题。这时多 Agent 的价值就开始显现，因为它能够把不同状态放进不同角色边界中管理。

再往系统治理看一层。只要任务开始需要缓存、重试、部分回滚、人工确认、版本对比，单体 Agent 的“一个上下文包打天下”就会越来越吃力。因为这些需求本质上都在要求系统能看清楚自己处在什么阶段、手上拿着什么状态、下一步应该由谁负责，而不是只会继续生成下一段文字。

这一段我们补足工程细节：长链路工具调用不是“多调几个 API”，而是在管理一组会变化的状态。搜索结果要不要缓存，失败工具要不要重试，中间结果能不能被覆盖，人工确认后是否形成新版本，这些都不是提示词能自然解决的问题。要注意：只要系统开始需要局部回滚、断点续跑、版本对比，就已经进入状态治理范畴。

2.1.5 【小结：什么时候必须考虑拆成多 Agent】

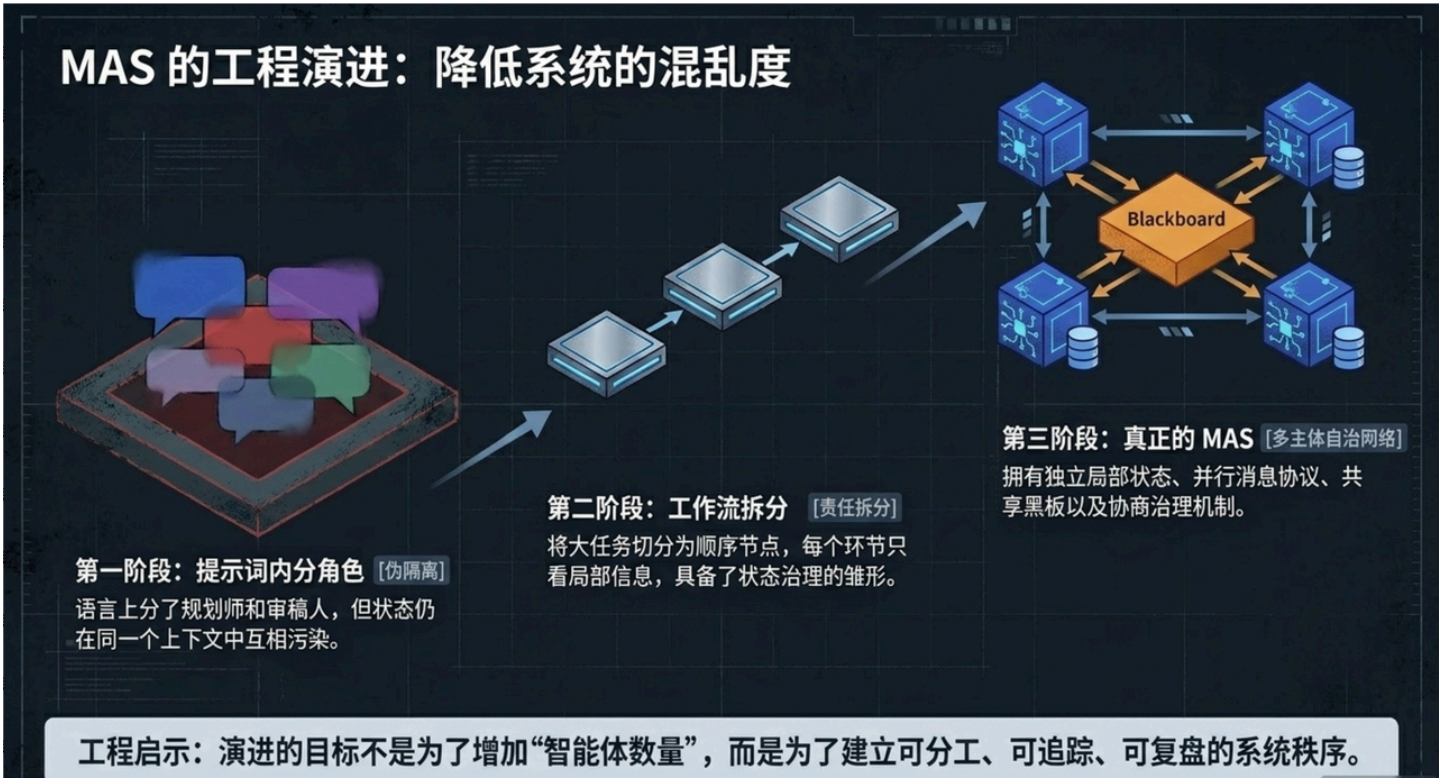
最后我们通过一句话来概括，单体 Agent 需要升级的信号通常有三种：任务链明显拉长，目标函数开始冲突，状态治理已经超出一个上下文能稳定承载的范围。注意这里说的是“稳定承载”，不是“偶尔能跑通”。很多单体 Agent 的问题，不在于它绝对不能做，而在于它不能持续、可复现、可审计地做好。

所以后面大家做架构决策时，不要问“一个 Agent 能不能勉强完成”，而要问“一个 Agent 能不能在可解释、可维护、可复盘的前提下稳定完成”。这两个问题的标准差别非常大。

可以用一个简短检查表来判断：一旦你发现系统开始大量依赖超长提示、越来越难定位错误、每次返工都要重跑整条链，那就说明它已经接近单体结构的舒适边界。看到这些征兆时，开始考虑 MAS 就不是激进，而是理性。

真正需要拆分的信号，通常不是任务听起来复杂，而是系统已经出现稳定性问题：上下文越来越长，错误越来越难定位，返工越来越依赖整体重跑，质量标准开始互相冲突。当出现这些信号时，升级单体 Agent 为 MAS 就有了较为充分的理由。

2.2 MAS 的演进逻辑



我们判断单 Agent 能力边界可以简化为三步：先看任务是否已经失稳，再定位失稳来自信息过载、目标冲突、角色混乱还是状态不可控，最后判断团队有没有足够的治理能力承接拆分。这不是讲课重复概念，而是通过实际工程化 AI 项目开发过程中的踩坑心得，提供给大家作为系统选型可以直接复用的判断路径。

2.2.1 【开场：从“一个全能执行器”到“多个专职角色”】

MAS 的演进逻辑可以理解为一个逐步分工的过程。最早大家会把所有能力都塞进一个提示词里，让一个 Agent 同时负责理解、规划、执行、反思。后来发现这种做法在简单任务上有效，但一碰到复杂链

路就容易失控，于是系统开始出现角色分离的需求。

这种分离本质上和软件工程里的模块化类似。模块化不是为了把代码切碎，而是为了降低耦合、提高可维护性。多 Agent 也是同理，它不是把“大模型调用”复制多次，而是把不同认知职责结构化拆开。

这和大家熟悉的软件工程经验很像：最开始一个脚本也能干很多事，但当同一个脚本同时负责配置、业务逻辑、异常处理和日志上报时，维护性会迅速恶化。多 Agent 系统的演进，本质上就是认知任务版本的“从大脚本走向模块化系统”。

这一步的关键，是把“全能”换成“专职”。全能执行器追求一次性解决所有问题，专职角色追求每个环节都有清楚责任。多 Agent 系统真正增加的不是热闹的程度，而是划分职责。

这个变化很像从个人工作台走向小型团队。个人工作台的优点是快，缺点是所有判断都压在一个上下文里；小型团队的优点是每个环节有负责人，缺点是必须设计交接规则。MAS 的讨论就是在这两个取舍之间展开。

2.2.2 【演进阶段一：提示词内部分角色】

这里介绍下 MAS 的第一个演进阶段——提示词内部分角色。

第一阶段其实还不是真正的 MAS，而是“单体内部角色化”。典型做法是在一个提示词里要求模型先扮演规划师，再扮演执行器，最后扮演审稿人。这个阶段的优点是成本低、上手快，不需要额外的消息流和状态管理。

但它的问题也很明显。角色虽然在语言上被区分了，状态却没有真正隔离。规划师生成的草案和审稿人的质疑都还堆在同一个上下文里，彼此会相互污染。所以这个阶段更像是“角色提示技巧”，还不是严格意义上的多主体系统。

这一阶段在教学上非常值得讲，因为很多团队都停留在这里。他们已经隐约感受到“分角色思考”有帮助，却还没有迈到系统级拆分。这里要明确：这一步不是错，而是不够。它适合作为过渡手段，但不能替代真正的角色边界和状态治理。

还有一种非常常见的过渡形态：团队先在同一条 Prompt 里写“你先作为规划师，再作为执行器，最后作为审稿人”。这一步有教学价值，因为它能让大家理解角色化思考；但它没有真正隔离状态，也没有独立责任边界，所以只能算单体 Agent 的自我分阶段，而不是 MAS。

这种阶段很适合作为入门，因为它让大家先看到“角色化思考”的价值。但它仍然有明显限制：所有角色共享同一个上下文，没有真正隔离状态，也没有独立的失败边界，所以它还不能算完整的 MAS。

2.2.3 【演进阶段二：工作流化与职责拆分】

下面介绍 MAS 的第二个演进阶段——工作流化与职责拆分。

第二阶段开始进入工程意义上的分工。系统把“规划、执行、复核、总结”拆成多个独立步骤，每一步由不同的子模块负责，模块之间通过结构化中间态连接。这时，即便底层还是同一个模型，系统层面也已经具备了“多角色、多状态”的雏形。

这个阶段的重要变化，不在于模型数量，而在于责任边界。每个环节只处理自己应该看到的信息，只输出自己应该负责的结果。错误定位开始变得可能，质量控制开始有了抓手，系统也能更自然地插入审计、回滚和人工确认。

工程上更直接的判断是：工作流化最大的价值，往往不是第一次成功率立刻提升很多，而是系统开始可以被诊断。可诊断性一旦建立，后续优化路径就会变得清晰。这也是为什么很多团队先从工作流分步做起，再逐步走向真正的 MAS。

这一段要强调工作流化的价值是“可诊断”。把规划、执行、复核拆成步骤后，即便底层仍然调用同一个模型，系统也能知道哪个阶段出了问题。比如计划阶段给出的步骤不完整，就不应该让执行阶段硬跑；复核阶段发现证据不足，也不需要整条链从头重做，而是可以把问题退回到资料补充节点。

工作流化之后，系统开始有阶段边界。规划、执行、复核不再混在同一段回答里，而是变成不同步骤。哪一步失败，就回到哪一步修，这就是从“会生成”走向“可诊断”的关键变化。

这一步虽然还不一定有多个独立 Agent，但已经开始接近 MAS 的思想。因为系统开始承认：复杂任务不是一个连续回答，而是一串有检查点、有输入输出、有返工位置的工作过程。

2.2.4 【演进阶段三：真正的多主体系统】

MAS 演进的第三个阶段才是严格意义上的 MAS。这里的多个 Agent 拥有相对独立的职责、局部状态、输入视角甚至目标函数，它们通过消息交换、共享黑板、任务市场以及协商协议共同完成更大的系统目标。这个时候，系统内部不再只是顺序步骤，而会出现并行、反馈、竞争和协商。

一旦进入这一步，系统设计问题就从“如何写好提示词”升级为“如何设计组织结构”。谁能看到什么，谁可以修改什么，谁来做最终裁决，失败如何回退，冲突如何解决，都开始成为一等问题。也正是在这个阶段，MAS 才真正显示出它和单体 Agent 的方法论差异。

这一段建议再补一个组织类比。一个真正的 MAS 更像一个小型项目团队，而不是一个能力更强的个体。项目团队最关键的问题，不是每个人都很聪明，而是谁负责什么、谁向谁汇报、冲突怎么仲裁、版本怎么同步。同样，MAS 的成熟度也主要体现在这些组织问题是否被设计清楚。

真正进入 MAS 后，关键变化是主体拥有局部状态和相对独立的目标。比如：研究员关心证据，规划师关心结构完整，审核员关心风险与可追责，协调者关心节奏和返工路径。它们更像一个小型项目团队：每个人角色边界都非常清晰，谁拥有决策权、谁能修改共享状态以及谁负责最终放行等等。

进一步讲，这一阶段最容易被误判的地方，是把“多个提示词节点”误认为“多个主体”。真正的主体至少要能保留自己的中间判断，并且它的判断会影响后续系统状态。例如研究员标记“证据不足”，规划师就不能无视这个标记继续生成完整结论；审核员判定“来源不可用”，协调者就必须触发返工，而不是把警告留在文本末尾。

因此，判断一个系统是否进入 MAS，可以看三个信号。第一，是否存在多个稳定角色，而不是临时的一次性步骤。第二，是否存在跨角色共享但不能随意覆盖的状态。第三，是否存在角色之间的反馈、驳回或协商。如果这三个信号都不存在，系统通常还停留在复杂工作流阶段。

真正的多主体系统会出现独立视角。研究员看到的是证据，规划师看到的是结构，审核员看到的是风险，协调者看到的是流程。每个主体都只掌握一部分世界，系统靠消息和状态把这些局部视角组织起来。

2.2.5 【小结：演进不是为了炫技，而是为了降低系统熵】

由此，我们可知，MAS 演进可以理解成一个降低系统熵的过程，相信大家在技术学习中都会碰到“熵”这个字，简单理解就是复杂性的意思。单体 Agent 在复杂任务中容易把不同阶段、不同目标、不同质量标准混成一团，熵会越来越高。多 Agent 并不是自动让系统变简单，而是通过分工、隔离和协调，把原本混乱的信息流重新结构化。

所以，演进的目标不是“更多主体”，而是“更低混乱度”。只要记住这句话，后面再讨论协作、竞争、协商时，大家就不会被表面形式带偏。

这一节可以总结成一个更容易记忆的结论，那就是单体 Agent 更像一个全能但容易过载的执行者，MAS 更像一个受组织规则约束的协作系统。前者追求单点能力，后者追求系统稳定性。理解这层差别，后续所有理论都会变得更顺。

MAS 演进的核心目标不是让系统看起来更复杂，而是让复杂任务变得更有秩序。单体 Agent 容易把事实、假设、计划、草稿混在一起，MAS 的价值就是把这些信息重新分层、分责、分流。

所以判断演进是否有价值，要看混乱度是否下降。拆分之后，如果事实更清楚、责任更清楚、错误更容易定位，演进就是有效的；如果只是调用次数更多、日志更长、接口更乱，那就是把复杂度换了一个地方堆起来。

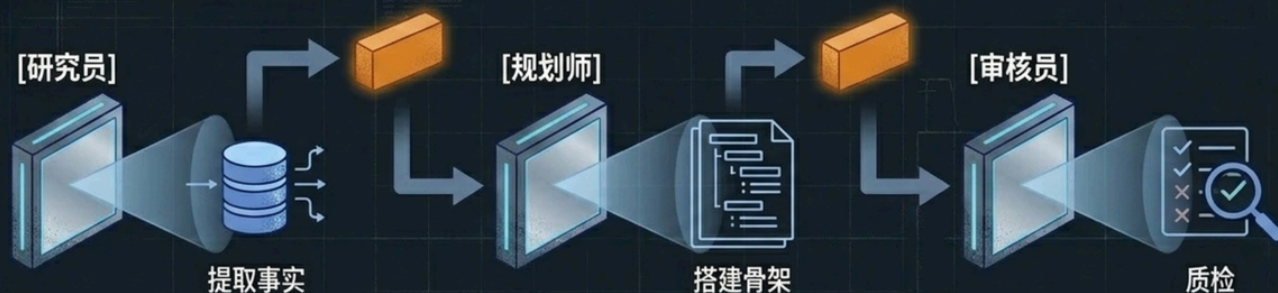
换句话说，MAS 的价值不是“多”，而是“清”。角色更清，状态更清，责任更清，失败路径更清。只要系统拆完以后这四件事没有变清楚，就说明这次拆分还没有真正成功。

第三部分：MAS 核心理论：协作、竞争、协商

3.1 协作机制

核心机制一：协作（Collaboration）

任务分解与认知互补



核心本质

将复杂的统一目标拆解为不同的“认知动作”。协作不等于完全并行，而是职责互补。每个角色只能看到自己该看的信息（局部状态）。

经典协作链

- [研究员] -> 提供带来源的事实清单（材料）
- [规划师] -> 搭建比较维度与章节结构（骨架）
- [审核员] -> 仅检查证据缺口与逻辑越界，不重写正文（质检）

工程警示：如果审核员开始自己补充素材，或者研究员开始撰写最终结论，协作即告破裂——责任边界已失效。

3.1.1 【开场：协作的本质是任务分解后的互补】

协作是 MAS 中最容易理解、也是最常见的一种机制。它的核心不是“一群人一起干活”，而是“通过任务分解，让不同主体分别处理自己最擅长的认知动作”。当一个复杂目标天然包含研究、规划、执行、评估等不同阶段时，协作就会比单体结构更加符合问题本身。

注意协作并不等于并行。很多协作系统实际上是串行推进的，只是每一段由不同角色负责。真正决定是否属于协作的，不是它们是否同时工作，而是职责是否互补、输出是否相互依赖、整体目标是否一致。

更落地地说，协作最适合处理“任何一个角色都掌握不了全部成功条件”的任务。只要成功依赖事实搜集、结构设计、质量审查这些不同认知能力的叠加，协作就不是锦上添花，而是问题结构的自然要求。

协作要讲清楚“互补”。多个 Agent 如果都在做同一件事，只是重复采样；只有当研究、规划、执行、审核分别承担不同认知动作时，协作才成立。比如：直接判断一个内容生产任务——创意 Agent、素材 Agent、审稿 Agent、发布 Agent 的职责是否真的互补。

协作最容易被误解成“多人同时做事”。更准确的说法是，不同角色处理不同认知动作。研究负责找事实，规划负责搭结构，执行负责产出，审核负责发现问题；这些动作互补，协作才有意义。

3.1.2 【原理拆解：角色、消息、共享记忆三要素】

多Agent系统的四大架构基石



角色边界 (Role Boundaries)

定义每个节点“负责什么”以及更重要的“不负责什么”。拦截越界操作。

消息协议 (Message Protocols)

摒弃大段无结构对话。采用“结构化字段 (Facts/Assumptions) + 自然语言解释”的接口规范。

共享记忆 (Shared State)

区分“局部草稿”与“全局黑板”。严格隔离“待验证假设”与“已确认事实”。

治理规则 (Governance)

定义终止条件、容错回滚路径与冲突仲裁机制。

架构底线：任何一项缺失，系统都会从“工程协作”退化为“无序群聊”。

一个可运行的协作系统，至少要回答三个问题。

1. 第一，角色边界怎么划分。研究员负责找事实，规划师负责定步骤，执行器负责落地，审核员负责挑错，这些边界必须明确，否则协作会退化成重复劳动。
2. 第二，消息如何传。消息如果只是自然语言长文本，容易丢结构；如果完全结构化，又可能过于僵硬，因此通常需要“结构化字段 + 自然语言解释”的折中形式。
3. 第三，也是最容易被忽略的一点，共享记忆怎么设计。协作不是所有 Agent 共享所有上下文，而是共享足够协作、但不过度污染的公共状态。比如任务目标、关键事实、版本号、待解决风险可以进入共享黑板；而每个角色的临时草稿、探索性思路、失败尝试则不一定需要全员可见。共享过少会导致彼此失联，共享过多又会带来上下文噪声，这正是协作系统设计的细活。

这里把三个要素和软件接口设计做一个类比。角色像模块边界，消息像接口协议，共享记忆像跨模块状态管理。任何一项没有定义好，系统都可能在运行中表现为“似乎每个角色都很聪明，但整体结果就是拼不起来”。这句话非常适合帮助理解理解，为什么 MAS 不是模型问题，而是系统问题。

再补一个容易被忽视的点，那就是共享记忆最好区分“可引用事实”和“待验证假设”。如果两者混在一起传播，后续角色很容易把假设当事实继续放大。现实系统里很多幻觉升级成系统级错误，恰恰不是因为单点幻觉，而是因为错误假设被多个主体接力传播。

三个要素可以理解成 MAS 的接口设计：角色边界回答“谁负责”，消息协议回答“怎么交接”，共享记忆回答“哪些信息全局可见”。如果共享记忆里把已验证事实、待验证假设、失败尝试全部混在一起，后续 Agent 就会把噪声当依据继续传播。这里可以按四类状态来区分：可共享事实、局部草稿、待验证假设和最终结论分别放在不同位置。

3.1.3 【案例：研究员、规划师、审核员的协作链】

下面我们用一个最小案例来解释 MAS 的协作。现在要做“AI 客服竞品研究报告”。研究员的任务是搜集公司列表、官网信息、产品能力和价格线索；规划师的任务是把研究目标拆成比较维度，例如客户类型、功能覆盖、商业模式、差异化能力；审核员的任务则不是重写报告，而是检查证据是否充分、结论是否越界、引用是否支撑论断。

这个案例里，三个角色的价值并不相同。研究员提供原材料，规划师提供结构，审核员提供质量约束。如果把三者都塞给一个 Agent，系统表面上也许能生成一份报告，但很难知道报告中的结构偏差到底来自研究不足、框架不合理还是审稿不严。协作链的优势就在于，它把问题定位能力也一起带进来了。

再往前一步看，协作链还会自然引出版本治理。研究员更新一条关键事实后，规划师是否需要重整框架，审核员是否要重新检查结论，都可以被设计成显式规则。这样系统就不再是“一次性回答”，而是一个有状态的、可迭代的工作流。

如果把这个案例讲完整，我们需要回答两个问题。

1. 第一，如果研究员拿到的资料彼此冲突，谁来决定哪条证据进入共享状态。
2. 第二，如果审核员判定“证据不足”，系统应该回退到研究阶段还是规划阶段。

大家一旦开始讨论这两个问题，就会意识到多 Agent 系统真正难在流程治理，而不是角色命名。

从交付视角看，客户真正购买的，从来不是“三个 Agent 在对话”，而是一份可以拿去开会、可以接受追问、可以定位问题来源的研究成果。只要这样表述，大家就会明白协作链的价值并不抽象，它直接对应可交付性。

我们把这个案例讲得更具体一些：研究员交付的是来源明确的事实卡片，规划师交付的是比较维度和章节骨架，审核员交付的是证据缺口、逻辑跳跃和风险清单。返工路径也要说清：如果审核员发现“价格判断没有来源”，问题应退回研究员；如果发现“比较维度不支撑结论”，问题应退回规划师。

案例讲解最后要回到验收：谁消费这个结果，如何判断它可用，出错后能否追溯到具体角色。只要验收对象和追责路径明确，案例就会从演示材料变成真正的工程样板。

这个案例可以拆成一条更清晰的数据流：研究员先输出事实卡片，每张卡片包含来源、可信度、适用范围和不确定性；规划师只允许使用这些事实卡片组织比较框架，不能凭空补结论；审核员再检查每个结论是否能反向追溯到事实卡片。这样一来，协作不是口头分工，而是由输入输出约束保证的分工。

这个设计还解决了一个真实项目里的常见问题：当最终报告出错时，团队可以定位错误发生在哪里。事实错了，回到研究员；结构错了，回到规划师；结论越界，回到写作或审核环节。MAS 的价值不只是“更多角色一起工作”，而是把问题定位能力提前设计进系统。

3.1.4 【风险与优化：协作系统为什么常常越拆越乱】

从以往 AI 项目结果来看，协作系统最常见的失败方式有三种。

1. 第一，角色职责没有真正分开，导致研究员也在写结论，审核员也在补素材，最后谁都能做一点，谁都不真正负责。
2. 第二，消息设计过于松散，上一角色输出大段自然语言，下一角色只能“凭理解接球”，信息损失严重。
3. 第三，共享状态没有版本控制，多个角色围绕不同版本的事实工作，最终结果彼此冲突。

优化方向也相应很明确。角色要按责任而不是按名词拆分，消息要尽量结构化，关键共享状态要有版本和来源标识。其实我们最终会发现，协作系统要做好的关键，从来不只是“大模型更强”，而是需要“组织工程清晰”。

这里再补一层会更有工程味：协作系统一旦变乱，团队通常会本能地继续加提示词、加规则、加提醒语，试图靠语言补丁把结构问题压住。但结构问题最终还是要靠结构解法。角色边界不清，就应该重画角色边界；共享状态混乱，就应该重构状态设计；接口松散，就应该重写消息协议。否则只是把复杂度从显性接口转移到隐性提示词。

3.1.5【小结：协作不是并行越多越好】

协作机制的底层逻辑我们最终可以压缩成一句话：把复杂目标拆成互补职责，再用消息和共享状态把它们重新缝合起来。这里面最难的地方不是“拆”，而是“缝”。因为一旦缝不好，系统就会出现接口断裂、责任漂移和质量失控。

所以一个成熟的协作系统追求的不是主体数量，而是主体之间的配合质量。角色少但边界清楚，往往胜过角色多却互相干扰。

判断是否该增加角色时，可以用一个极简原则：每增加一个角色，都必须说明它减少了什么混乱、承担了什么责任、引入了什么成本。能答出这三件事，协作就有必要；答不出来，角色大概率只是装饰。

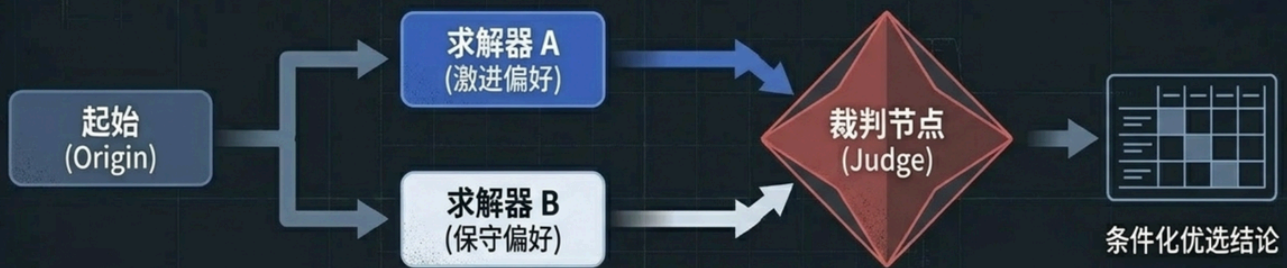
协作的边界也要说清楚。并行越多，消息成本也越高；角色越多，协调难度也越大。好的协作不是把任务切得越碎越好，而是切到刚好能降低混乱、暴露问题、提升可控性。

一个稳定的协作系统，通常会保留少量关键角色，而不是无限增加角色。每个角色都必须有独立产物和独立验收标准，否则它只是换了一个名字参与聊天，并没有真正承担责任。

3.2 竞争机制

核心机制二：竞争（Competition）

引入系统级差异，对抗高不确定性



核心本质

保留多个独立候选方案，对抗单一路径的过早收敛。输出的不仅是胜者，更是“比较价值”。

如何构造有效竞争

- 输入差异：分配不同的信息视角（如市场资料 vs 财务资料）。
- 偏好差异：设定冲突的目标函数（如高速扩张渗透 vs 保守现金流）。
- 显式评估：必须有一个基于明确维度（成本、周期、风险）的裁判节点。



避坑指南：没有差异性设计和明确评审维度的并行生成，只是在无意义地浪费 Token。

3.2.1 【开场：为什么系统里要允许竞争性方案存在】

如果说协作强调的是“分工互补”，那么竞争强调的就是“多解并存，再择优而取”。在很多高不确定任务中，问题不在于没有答案，而在于答案空间太大、单一路径太容易陷入局部最优。此时让多个 Agent 以不同假设、不同策略、不同视角并行求解，反而能提升最终质量。

这传统搜索算法里的搜索、集成学习里的多模型投票有某种精神上的相似性。不同之处在于，MAS 中的竞争主体往往不仅输出结果，还输出推理路径、证据选择和策略偏好，因此竞争的不只是结论，也包括“怎么得出这个结论”。

这里建议再补一个很实用的提醒：竞争机制最有价值的地方，不是证明“一个答案对，一个答案错”，而是强迫系统把不同策略的差异显式暴露出来。很多管理决策最怕的不是分歧，而是分歧被隐藏在一个看似完整的答案里。竞争恰恰能把这种隐性分歧结构化呈现。

竞争机制的价值在于保留分歧。很多复杂决策最怕系统过早给出一个看似确定的答案，而竞争性方案能让不同假设先并排出现，再进入比较和评审。

3.2.2 【机制拆解：候选解生成、评分与淘汰】

一个竞争系统通常包含三个环节。

1. 第一，生成多个候选方案。这些候选必须具有真实差异，不能只是同义改写。

2. 第二，建立评分或比较机制。评分可以来自另一个评审 Agent、规则函数、外部指标，或者人机混合判断。
3. 第三，进行淘汰或合并，输出最优方案或一个经过吸收后的综合方案。

这里的关键不在于“多生成几个结果”，而在于多样性是否有意义。如果两个 Agent 接收到完全相同的上下文和目标，又没有策略差异，它们只是重复劳动。要让竞争有效，必须在输入、角色目标、评估尺度或求解策略上刻意制造差异性。

再往下拆一层，差异性可以这样设计出来。常见方法包括：给不同主体不同优先级目标，例如增长优先和利润优先；给不同主体不同信息视角，例如市场侧资料与财务侧资料；给不同主体不同约束，例如预算受限和时间受限。只有差异被显式编码进系统，竞争才不会退化成“多抽几次卡”。

此外，评分机制本身也必须可解释。评审 Agent 如果只说“方案 A 更好”，却说不清评分维度和依据，那么竞争系统就会重新掉回黑箱。真正成熟的竞争机制，不只是选出赢家，还要保留比较理由，方便后续审计和复盘。

竞争机制要讲清楚“差异从哪里来”。可以通过不同目标制造差异，例如利润优先、增长优先、风险优先；也可以通过不同信息视角制造差异，例如市场资料、财务资料、用户访谈分别进入不同求解器。没有差异设计，多生成几个答案只是重复采样，不是真正的竞争。

3.2.3 【案例：两个求解器竞争生成市场进入策略】

假设我们要为一家新进入 B2B AI 客服市场的创业团队制定进入策略。我们可以设置两个竞争性 Agent:

1. 第一个 Agent 采用“保守盈利路径”，优先寻找付费意愿高、实施阻力低的垂直领域。
2. 第二个 Agent 采用“高速渗透路径”，优先考虑容易形成网络效应或品牌的切入口。

两者使用相同的基础事实，但目标偏好不同。

随后再引入一个评审 Agent，按照市场容量、切入难度、销售周期、产品差异化空间四个维度给两份方案打分。这个时候，竞争机制的价值就体现出来了。系统不再过早收敛到单一路径，而是通过显式比较，把“策略分歧”转化为可审议的结构化差异。对管理者来说，这种输出要比一个唯一答案更有决策价值。

更像真实业务的做法，是让评审 Agent 输出一个“条件化结论”：当团队融资充足、允许更长回收周期时，偏高速渗透方案；当团队现金流压力大、需要尽快形成付费样板时，偏保守盈利方案。这样大家会看到，竞争机制最终不一定只产生一个唯一答案，也可能产出一套带条件的决策建议。

更工程化地说：竞争系统的真正产物，往往不是“赢者通吃”，而是更高质量的比较材料。只要这层价值讲透，大家就不会把竞争误解成简单的胜负游戏。

3.2.4 【边界：竞争何时提升质量，何时只是烧 Token】

竞争机制并不是免费午餐。它会显著增加调用次数、评审成本和治理复杂度。如果任务本身低不确定、规则明确、单一路径已经足够稳定，那么引入竞争只会带来 Token 消耗和系统噪声。比如简单 FAQ、固定模板归纳、明确规则抽取，这类任务通常没有必要搞多主体竞争。

竞争真正适合的，是那些“存在多个合理解”“评价标准需要权衡”“错误代价不允许单一路径拍脑袋”的任务。也就是说，竞争机制适合高不确定、高收益、高比较价值的场景，而不是所有复杂任务。

这里还应该补一个常见误区：有些团队把“多生成几版候选”误当成竞争机制，但没有任何系统化比较方法，最后只能靠人工主观挑一个看起来顺眼的版本。这种做法当然有时也能用，但它不是完整的多Agent竞争。完整竞争至少要有差异设计、评审标准和结果归档这三个环节。

这一段适合补一个成本视角：竞争机制会增加调用次数、评审复杂度和结果合并成本。只有当问题存在多个合理解，并且错误选择的代价足够高时，竞争才值得。简单 FAQ、规则抽取、固定格式总结通常不需要竞争，因为它们的关键是准确召回和规则约束，不是多方案比较。

3.2.5 【小结：竞争适合高不确定、高收益任务】

竞争的本质，是通过保留多个候选方案来对抗过早收敛。它不是为了让系统更热闹，而是为了让系统在关键决策点拥有更强的比较能力。只要大家记住“竞争一定要有真实差异和明确评审”，就不容易把它误用成简单的重复采样。

一句话总结竞争机制，那就是当系统面对的是开放问题，且不同路径可能带来明显收益差异时，竞争值得引入。它不是为了制造更多文本，而是为了制造更好的比较。

在低不确定任务里，竞争没有必要；在高不确定任务里，竞争能帮助系统保留多种可能。关键是要有评审规则，否则候选越多，决策者越累，系统反而失去收敛能力。

所以竞争机制要少用，但要用在关键处。普通确定性任务不需要竞争，战略选择、方案评估、复杂决策才需要竞争。把竞争留给真正值得比较的节点，系统才不会被候选方案拖慢。

3.3 协商机制

核心机制三：协商（Negotiation）

约束边界下的冲突收敛与资源排程



```
协商收敛算法:  
while not consensus and loop < max_rounds:  
    propose(preferences)  
    evaluate_constraints(hard_limits)  
    if acceptable:  
        yield()  
    else:  
        concede_based_on_rules()  
return finalized_schedule
```

1. 硬约束 (Hard Constraints)

绝对不可突破的系统底线
(如API额度、最高预算)。

2. 偏好 (Preferences)

可让渡或妥协的目标倾向
(如优先出图 vs 优先查资料)。

3. 终止规则 (Termination)

达成共识、最大轮次超时或单
方拒绝让步时的仲裁判定。

3.3.1 【开场：协商为什么比协作更难】

协作和竞争都相对容易理解，因为它们要么共享总目标，要么比较候选方案。协商更难，是因为它面对的是多主体之间的约束冲突。不同 Agent 可能各自代表不同偏好、资源边界或评价指标，系统不能只靠“谁声音大听谁的”，而要通过规则化过程达成一个可接受的共识。

换句话说，协商处理的不是知识不够，而是偏好不同。只要存在资源有限、时间有限、风险预算有限、收益目标不同这类约束冲突，协商机制就有出现的必要。

这也是为什么协商看起来不像“智能生成”，却常常决定系统成熟度。很多真实系统里，真正卡住流程的不是没人能产出内容，而是谁都能提出合理诉求，却没有机制决定先满足谁、满足到什么程度、在哪个边界上停止让步。协商机制就是为这种复杂度而生的。

协商比协作难，是因为它处理的不是简单分工，而是利益、偏好和约束的冲突。协作里大家通常朝同一个方向推进；协商里，每个主体都可能有自己的底线和让步空间。

协商还会引入一个额外问题：主体之间可能都没有错，只是目标不一致。一个角色追求速度，另一个角色追求稳定，第三个角色追求成本最低。系统要解决的不是谁对谁错，而是如何在约束下形成共同方案。

3.3.2 【原理：偏好冲突、约束交换与共识形成】

协商系统至少包含三类信息。

1. 第一类是主体偏好，比如某个 Agent 更重视成本，另一个更重视速度。
2. 第二类是硬约束，比如预算不能超过多少、交付时间不能晚于何时、风险等级必须低于某个阈值。
3. 第三类是可让渡空间，即哪些条件可以交换、哪些底线不能动。

协商过程通常不是一次性完成的，而是经过多轮提案、反提案、让步和确认。每一轮都在更新系统对“可接受解”的边界理解。因此协商机制设计的重点，不只是消息格式，还包括终止条件、让步规则和共识判定标准。没有这些，所谓“协商”很容易退化成无穷无尽的来回对话。

更细的一点是：硬约束最好与偏好显式分开编码。因为硬约束一旦被当成可让渡条件，系统就可能在不知不觉中违反底线；反过来，如果偏好也被写成硬约束，协商空间又会被过早锁死。很多协商失败，本质上不是主体太聪明，而是约束类型没有分清。

另外，终止条件也非常重要。系统需要知道是在“找到一个满足底线的方案时停”，还是“达到最大轮次时停”，还是“任何一方拒绝再让步时停”。没有终止规则，协商不会变得更聪明，只会变成更昂贵的循环。

协商原理要把硬约束和偏好分开。硬约束是不可突破的底线，例如预算上限、合规要求、截止时间；偏好是可以权衡的倾向，例如更快、更便宜、更稳。若把偏好写成硬约束，系统没有让步空间；若把硬约束当偏好，系统可能生成违反底线的方案。

协商的核心不是让所有主体都满意，而是在约束允许范围内找到一个可执行解。真实系统中经常会出现“局部最优互相冲突”的情况：销售 Agent 想尽快承诺客户，交付 Agent 担心资源不足，财务 Agent 关注毛利率，风控 Agent 关注合规边界。协商机制要做的，是让这些主体把不可突破的底线和可以交换的偏好说清楚。

协商过程可以拆成四个动作：声明立场、暴露约束、提出让步、确认共识。声明立场解决“我想要什么”，暴露约束解决“我不能接受什么”，提出让步解决“我愿意交换什么”，确认共识解决“最终按什么执行”。只要缺少其中一步，协商就容易变成空泛对话。

3.3.3 【案例：资源有限条件下的任务排程协商】

假设一个内容工厂系统里同时存在三个任务：行业研究简报、营销海报生成和客户答疑脚本更新，但今天只有有限的 API 预算和人工审核时长。研究 Agent 希望优先做研究，因为它影响后续多个任务；营销 Agent 希望先出海报，因为活动马上开始；客服 Agent 则强调脚本更新直接影响在线转化。此时问题已经不是谁更会干活，而是谁应该先获得有限资源。

协商机制会要求各主体把“请求理由”结构化表达出来，例如业务影响、紧迫度、资源消耗、失败后果。然后由一个协调 Agent 或规则引擎根据这些信息做折中安排。这个案例非常典型，因为现实业务中的多 Agent 系统，很多时候真正难的不是生成内容，而是决定“谁先做、谁后做、谁让步”。

更完整的讲法是加入一个动态变化：在协商进行到第二轮时，突然来了一个高优先级客户投诉，需要立即更新客服脚本。此时原有排序是否要被打断，哪个任务应该让步，协调器有没有权力重新分配预

算。大家一旦讨论这种动态插队场景，就会更加理解协商为什么不是一次静态排序，而是一种持续的组织能力。

还有一个现实提醒：很多业务场景真正购买的，不是“最优答案”，而是“在资源有限时还能给出可解释的折中方案”。这正是协商机制相较于纯生成模型的管理价值。

这个案例可以加入动态变化：原计划先做行业研究和营销海报，但客服侧突然出现高优先级客户投诉，需要立即更新答疑脚本。协调 Agent 必须重新计算紧急度、影响范围和资源占用，并说明为什么某个任务被延后。这样大家能看到协商不是静态排序，而是在约束变化中维持可解释的折中。

案例讲解最后要回到验收：谁消费这个结果，如何判断它可用，出错后能否追溯到具体角色。只要验收对象和追责路径明确，案例就会从演示材料变成真正的工程样板。

为了让这个案例更落地，我们也可以给每个任务设置明确字段：业务影响、截止时间、预计成本、失败后果、可延期程度。研究简报可能影响后续多个决策，但未必当天必须完成；营销海报可能紧急但影响范围较窄；客服脚本更新可能直接影响客户体验，且出现高优先级投诉后会立刻改变排序。字段一旦结构化，协商才有依据。

协调 Agent 的输出也应该是可解释排程，而不是一句“先做客服”。它需要说明先做什么、延后什么、释放什么资源、触发什么人工确认，以及如果新事件再次进入系统，排序规则是否重新计算。这样大家能看到，协商机制的交付物是“带理由的调度决定”。

3.3.4 【机制示例：一个极简协商循环伪代码】

下面我们通过一个极简伪代码帮助理解协商循环，不是生产级实现。重点不是语法，而是看清楚：每个主体都要显式提交目标、约束和最低接受条件，协调器再在多轮交互中寻找可行解。

代码块

```
1  agents = [  
2      {"name": "research", "priority": 9, "cost": 4, "deadline": 2},  
3      {"name": "marketing", "priority": 8, "cost": 3, "deadline": 1},  
4      {"name": "support", "priority": 7, "cost": 2, "deadline": 1},  
5  ]  
6  
7  budget = 6  
8  selected = []  
9  
10 while agents and budget > 0:  
11     agents.sort(key=lambda x: (x["deadline"], -x["priority"]))  
12     proposal = agents.pop(0)  
13     if proposal["cost"] <= budget:  
14         selected.append(proposal["name"])  
15         budget -= proposal["cost"]
```

```
16     else:
17         proposal["priority"] -= 1
18         if proposal["priority"] > 0:
19             agents.append(proposal)
20
21     print(selected, budget)
```

这段代码故意做得很简化，但它已经展示了协商系统的三个核心问题：

1. 谁提交诉求
2. 诉求如何被比较
3. 资源不足时如何让步

真正的 MAS 框架只是在这个骨架之上，把消息协议、状态持久化和回合控制做得更完整。

代码讲解可以补三点：

1. priority 表示业务价值
2. cost 表示资源消耗
3. deadline 表示时间压力。

循环里的排序其实就是一个简化的协商规则，先处理时间压力更高、优先级更高的请求。

真实系统还需要日志、拒绝理由、人工仲裁和最大轮次，否则很容易陷入反复让步或无限重排。

看这段伪代码时，不要只看循环本身。真正重要的是每一轮循环都在收集立场、更新约束、缩小可接受解空间。协商不是聊天，而是不断逼近一个能执行的折中结果。

3.3.5 【小结：协商是让系统学会“带约束地达成一致”】

协商机制的价值，在于它把原本隐性的冲突显性化。系统不再假装所有目标天然一致，而是承认主体之间可能存在拉扯，然后用规则化过程去收敛。这一点非常重要，因为现实中的复杂系统，很多失败都不是因为模型不会想，而是因为系统没有能力处理内部冲突。

所以协商可以被理解成一种“冲突治理能力”。当任务走到资源分配、优先级排序、风险权衡这些层面时，协作和竞争往往都不够，协商才是真正决定系统成熟度的那一步。

最后可以用一个简短判断进行总结：只要系统里开始出现“都重要，但不能都先做”“都合理，但不能都满足”这种句式，协商问题就已经出现了。能识别到这一点，往往比会写一个复杂协商协议更重要。

协商的最终目标，不是让所有主体都满意，而是在硬约束不被突破的前提下形成可执行方案。只要存在资源有限、时间有限、目标冲突，协商就会从可选能力变成系统必需能力。

因此，协商机制一定要保留约束记录。每一次让步、拒绝和接受，都应该能回溯到具体理由。只有这样，最终方案才不是一段折中话术，而是一个能解释、能执行、能复盘的决策结果。

这一点放到真实业务里尤其重要。很多时候，系统不是没有答案，而是答案之间互相牵制：想更快就要花更多预算，想更稳就要牺牲速度，想更省就要接受风险。协商机制就是把这些牵制关系显式处理掉。

第四部分：应用领域分析

4.1 典型应用领域

4.1.1 【开场：不要按行业先看，要按任务结构先看】

应用领域分析最容易犯的错误，是一上来按行业切，比如金融、医疗、教育、电商。行业分类对业务沟通有帮助，但对系统设计帮助有限。因为同一个行业内部，既可能有适合单体 Agent 的简单任务，也可能有必须依赖 MAS 的复杂场景。

更有工程价值的视角，是看任务结构。只要任务天然包含多角色、多阶段、多状态、多评价标准，它就更有可能适合 MAS。反之，如果任务路径短、输出标准固定、错误容易校验，那就未必需要多主体。

这里最好再补一个判断标准：行业只是场景外壳，任务结构才是系统骨架。外壳相似并不意味着架构相似。只要大家建立起这个区分，后面遇到任何新行业，他们都能先问“这里到底存在什么角色和什么状态”，而不是先被行业标签带着跑。

同一个行业里可能同时存在适合 MAS 和不适合 MAS 的任务。比如客服行业里，简单 FAQ 不适合拆；复杂投诉协调可能适合拆。判断入口一定是任务结构，而不是行业标签。

这个判断能避免很多误用。看到金融、医疗、客服、代码这些标签时，不要马上判断适不适合多 Agent；先拆任务，看里面是否真的存在多个稳定角色、多个状态来源和多个质量标准。

4.1.2 【领域一：研究与情报分析】

研究与情报分析是最典型的 MAS 适用领域之一。原因在于它天然包含资料搜集、来源验证、框架整理、结论撰写和风险审稿多个角色。单体 Agent 可以写一份“看起来像研究报告”的文本，但要稳定地产出带证据、带结构、可复核的分析结果，多角色分工会明显更稳。

尤其在需要形成决策支撑材料时，研究类任务很适合引入“研究员 + 规划师 + 审稿人”的协作链，必要时再加入竞争性方案生成，用不同策略并行形成备选判断。

我们继续往业务价值上看一步：研究类任务的输出通常会被二次消费，比如拿去开管理层会议、拿去支持产品决策、拿去形成商务判断。只要输出会被追问，系统就不能只给一个看起来顺滑的结论，而必须保留证据路径和责任链条。这也是研究场景特别适合 MAS 的核心原因。

研究类场景要强调证据链。一个能写报告的 Agent 并不等于能交付研究系统；研究系统必须知道资料来自哪里、哪些结论只是假设、哪些判断可以被引用到管理层会议。多 Agent 在这里的价值是把资料搜集、框架整理、结论生成、证据审查拆开，使报告可追问、可复盘。

研究场景还特别适合展示“并行与复核”的组合。多个研究 Agent 可以分别检索官网、新闻稿、招聘信息、产品文档和第三方评论，再由来源审核 Agent 判断哪些信息可以进入正式报告。这样做虽然调用更多，但能显著降低单一来源偏差，也方便在报告末尾保留证据索引。

需要提醒大家的是，研究类 MAS 不是把同一个问题问三遍，然后投票选答案。更成熟的做法是让不同角色承担不同证据渠道或不同分析角度，再用统一的证据标准合并。否则多 Agent 只是在扩大噪声，而不是提高研究质量。

4.1.3 【领域二：软件开发与代码协同】

代码协同同样非常适合 MAS。一个复杂开发任务至少包含需求理解、方案设计、代码生成、测试验证、日志分析和质量审查等步骤。把所有动作交给一个 Agent 虽然也能做，但在长链路任务里经常出现“写得快、测得浅、修改错”等问题。

因此，多 Agent 代码系统往往会出现需求分析 Agent、实现 Agent、测试 Agent、审查 Agent、环境执行 Agent 等角色。这也是为什么后续 AutoGen、CrewAI、LangGraph 在代码协作场景里会格外活跃，因为它们天然适合承接这种角色化结构。

看一个贴近开发团队的例子：真正难的不是让模型写出一段函数，而是让系统在修改一处逻辑后，知道哪些测试要重跑、哪些日志要检查、哪些模块可能被波及。只要任务进入这种“变更影响管理”层面，多角色协同就会比单点生成更像真实工程。

软件开发场景可以补一个工程链路：需求分析 Agent 明确验收条件，方案 Agent 给出改动边界，实现 Agent 写代码，测试 Agent 运行用例，审查 Agent 查回归风险。真正难的不是生成一段函数，而是修改之后知道哪些模块受影响、哪些测试该重跑、哪些日志能说明问题。

代码部分需要说明一个边界：教学代码只表达结构，不表达生产能力。生产系统还需要异常处理、版本持久化、权限控制、模型调用日志和人工确认点。

软件开发场景天然包含多个质量视角。需求理解、代码实现、测试验证、日志分析、变更总结都可以拆成不同责任。拆分之后，系统更容易知道错误来自需求理解、实现细节还是测试覆盖不足。

4.1.4 【小结：适合 MAS 的任务通常具有三重特征】

综上，适合 MAS 的任务通常具有三重特征。

- 1. 第一，任务天然可以拆成不同职责，而且拆分后不是形式主义。
 - 2. 第二，多个角色之间存在真实的信息交换或目标权衡。
 - 3. 第三，系统结果的质量不仅取决于单次生成，还取决于过程中的协调和治理。
- 只要这三条不成立，即使任务看起来“很复杂”，也未必值得上多Agent。因为复杂不等于适合分工，有时候复杂只是数据脏、规则乱、需求不清，这类问题先靠治理，不先靠多主体。
- 把这三重特征记住之后，面对新需求时就不用先想框架，而是先看结构。只要角色、状态、质量标准三件事同时变复杂，多Agent才有讨论价值。

4.2 场景选型与反例

4.2.1 【开场：不是所有复杂任务都值得上多 Agent】

机制诊断矩阵：正确匹配你的系统难题			
维度	协作 (Collaboration)	竞争 (Competition)	协商 (Negotiation)
核心难题	分工与信息缝合	避免局部最优，多解择优	资源分配与冲突收敛
关键要求	严格的角色边界与接口规范	输入/目标的显式差异设计	区分硬约束与可让渡偏好
典型输出	组装好的最终流转产物	胜出方案及对比评估理由	带妥协条件的资源排程计划
失效风险	接口断裂，责任互相推诿	趋同生成，无意义烧 Token	陷入无限对话的死循环

这里需要再次重申，不是所有复杂任务都值得上多 Agent，因为很多团队一旦学会多Agent，就容易产生“万物皆可多 Agent”的冲动。实际上，复杂任务也分两种。一种复杂是“任务结构复杂”，确实需要分工协调；另一种复杂是“问题定义混乱”，这时上多少 Agent 都只是在放大混乱。

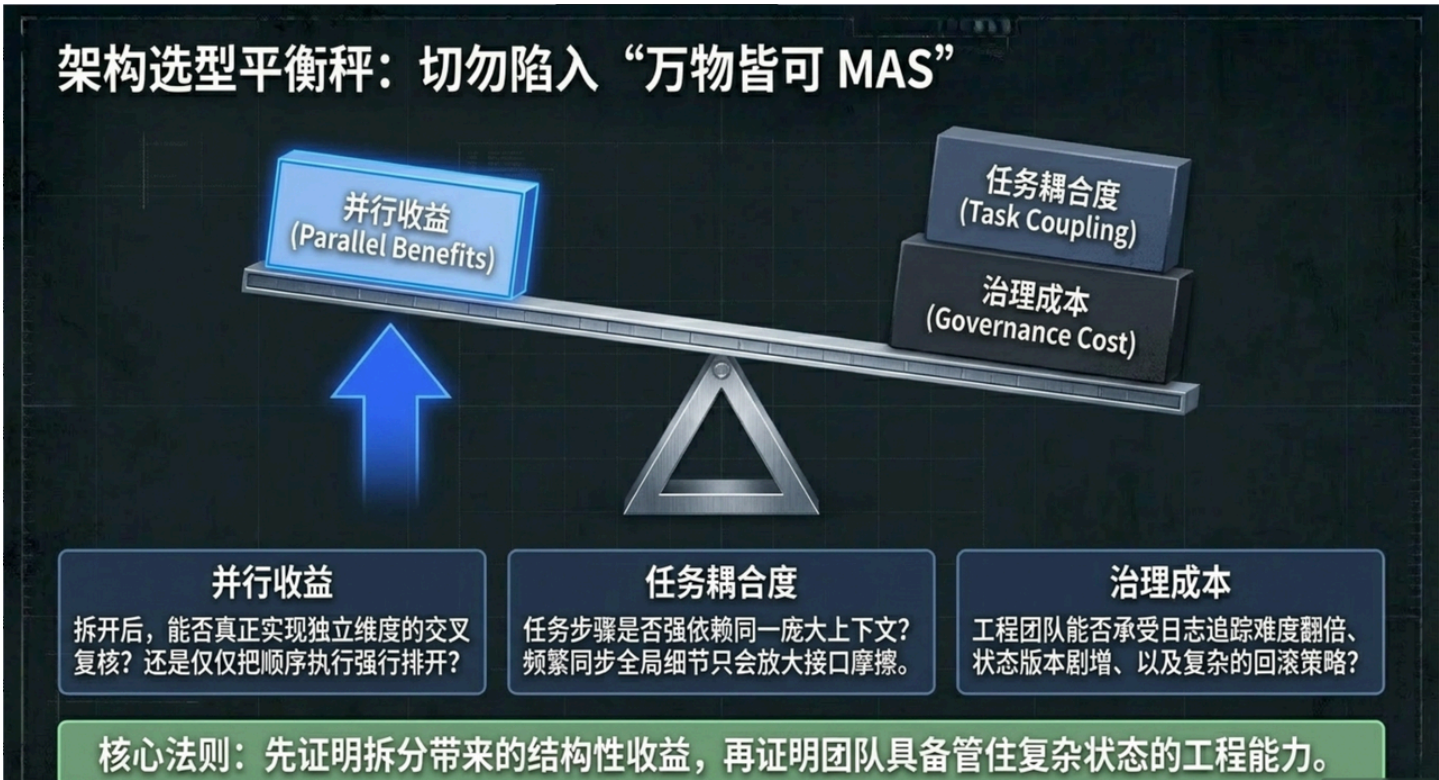
所以做选型时，第一步不是想象能拆出多少角色，而是先问：这个任务的复杂度来自哪里。是来自多角色分工，还是来自需求本身还没被澄清。

还有一句非常适合客户沟通的话：如果需求连成功标准都说不清，多 Agent 不会帮你把问题变清楚，它只会把模糊放大成系统噪声。先澄清目标，再决定是否拆角色，这个顺序不能反。

复杂不等于适合 MAS。有些任务复杂，是因为数据脏、规则乱、目标没说清；这种复杂应该先治理输入，而不是增加主体。否则系统只是把原来的混乱放大。

真正的工程成熟度，往往体现在会拒绝过度设计。能用检索解决的问题，不要用协作解决；能用规则解决的问题，不要用协商解决；能用单体稳定完成的问题，不要为了展示架构而拆成多主体。

4.2.2 【判断框架：任务耦合度、并行收益、治理成本】



判断场景是否需要多 Agent，我们可以看三件事。

1. 第一，任务耦合度。如果各步骤强依赖同一份上下文、每一步都必须回看完整历史，那么拆分后未必真的能减少复杂度。
2. 第二，并行收益。如果拆分之后能够让不同角色并行探索、交叉复核或形成多解比较，那就有明显收益。
3. 第三，治理成本。如果角色一多就必须引入复杂的状态管理、消息协议、日志系统和回滚机制，那么收益必须足够大才值得承担这些成本。

这三项判断其实对应三个很现实的问题：拆开之后会不会更清楚，会不会更快，会不会更难维护。只要其中两项答不上来，就不要急着上 MAS。

这里最好再展开一下任务耦合度。假设一个任务每一步都必须依赖前面所有细节，而且中间态很难结构化，那么强拆角色之后，大家仍然不得不反复共享整段上下文，此时分工收益就会迅速被接口成本

吃掉。反过来，如果任务天然可以切成“找事实、做判断、审风险”这样的层次，拆分就会非常自然。

治理成本则是很多团队最容易低估的部分。角色数量增加，意味着日志链条更长、错误定位更难、状态版本更多、回滚策略更复杂。如果没有相应的工程能力，理论上的分工优势很可能被管理成本抵消掉。所以判断 MAS 是否值得，永远要把“能不能管住”放在“能不能做出来”旁边一起看。

第五部分：课程总结与课后任务

6.1 核心要点回顾

6.1.1 【回顾一：多 Agent 的出现是工程分工的结果】

今天的 MAS 理论介绍很干，很难。如果大家听得云里雾里，建议线下再复盘下文档资料加深理解。

最后我们回顾和总结下结论：

今天第一条最重要的结论是，多 Agent 系统不是从概念爱好出发，而是从工程压力出发。只要任务开始出现长链路、多目标、强状态治理需求，单体 Agent 就会逐渐暴露边界，多主体分工才会变得必要。

这句话请大家一定记牢，因为它决定了后续所有框架学习的方向。不是因为框架能做多 Agent，所以我们要找场景去套；而是因为场景确实要求分工，所以我们才去找合适框架。

最后回看一下今天的主线：今天前面讲的上下文膨胀、角色冲突、状态治理，其实都在指向同一个结论，即系统复杂度已经不能再靠单点提示词硬扛。只要把这条主线再串一次，整节课的逻辑会更闭合。

6.1.2 【回顾二：协作、竞争、协商对应三种系统逻辑】

第二条结论是，协作、竞争、协商分别对应三类不同问题。协作解决分工互补，竞争解决多解优选，协商解决冲突约束下的共识形成。真实系统里三者往往会混用，但只有先分清逻辑，才能判断某个场景究竟缺的是哪一种机制。

这也是为什么 MAS 理论不能被简化成“多个 Agent 一起聊天”。如果只看到表面互动，看不到底层机制，系统设计就会很容易失焦。

课后大家可以把自己熟悉的一个业务流程分别套到这三个机制上去思考：哪里需要协作，哪里值得竞争，哪里必须协商。这个动作一做，理论就不再停留在定义层，而会转成可迁移的方法。

这三种逻辑可以直接对应三类问题：事情太多，需要协作；路径不确定，需要竞争；目标有冲突，需要协商。把问题类型判断清楚，机制选择就不会乱。

这三个词也可以作为调试入口。协作出问题，优先看角色边界；竞争出问题，优先看评分标准；协商出问题，优先看硬约束和让步规则。这样机制不只是概念，也能指导排错。

6.1.3 【回顾三：应用判断先看收益，再看治理】

第三条结论是，应用判断要先看拆分收益，再看治理成本。并不是主体越多越先进，也不是任务越复杂越适合 MAS。只有当任务结构、并行收益、质量要求和治理能力共同成立时，多Agent才真正有价值。

用更短的话说，就是“先证明为什么值得拆，再证明拆了能管住”。这个判断习惯，比背多少框架 API 都重要。

这里最好再补一句面向真实项目的话：很多架构问题最后不是败在“做不到”，而是败在“做得起但管不起”。多Agent 系统尤其如此。只要团队没有把治理成本放进决策里，前期的架构兴奋感往往会在后期维护中被成倍反噬。

回顾第三点，应用判断要先看收益，再看治理。收益说不清，多Agent就是装饰；治理跟不上，多Agent就是风险。真正成熟的设计，一定同时考虑这两边。

收益和治理之间要保持平衡。收益告诉我们为什么值得拆，治理告诉我们拆完以后能不能管住。只讲收益会乐观，只讲治理会保守，两者合在一起才是工程判断。

6.1.4 【一句话收束】

最后我们再把整节课压缩成一句话，那就是：多 Agent 系统的本质，不是让更多主体说话，而是让复杂任务在分工、比较、协调和治理中变得可控。

6.2 课后作业

6.2.1 【作业说明】

今天课后请大家任选一个自己熟悉的业务需求，例如竞品研究、代码修复、内容生产、营销活动规划等，然后完成三个动作。

1. 第一，用不超过 150 字说明这个需求为什么可能适合或不适合多Agent。

2. 第二，若适合，请至少拆出 3 个角色，并写清楚每个角色的输入、输出和责任边界。
3. 第三，再补一段说明：这个系统最需要的是协作、竞争还是协商，为什么。

6.3 下节课预告

6.3.1 【预告：从理论走向 AutoGen 多智能体框架】

下一节课我们会从今天的理论基础正式走向 AutoGen。重点不是简单跑通 Demo，而是看一个多智能体对话框架如何管理角色、消息、回合与协作。大家可以带着今天的四个关键词进入下一讲：分工、消息、状态、治理。只要这四个词没有丢，框架学习就不会变成只记接口的表层操作。

04

附录：教学资源

4.1 推荐阅读

- 《Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence》
- Contract Net Protocol、Blackboard Model、Task Allocation 的基础资料
- AutoGen、CrewAI、LangGraph 官方文档中的多角色编排章节
- 与组织理论、博弈论、分布式系统协调机制相关的入门材料